

Algoritmo de Diseño Generativo Bio-Físico

Gierer-Meinhardt Anisotrópico con Gradientes Radiales

Brief Técnico de Implementación Matemática y Algorítmica

Proyecto: FungalMorphoSpace $\rightarrow$ BioPhysical Core

Autor Original: Mario Ahumada Durán

Análisis Técnico: Claude (Anthropic)

Versión: 1.0

Fecha: Enero 2026

Declaración de Innovación

“No usamos generación procedimental de laberintos. Usamos Difusión Anisotrópica de Gierer-Meinhardt que garantiza alineación termodinámica con vectores de carga mecánica.”

Índice

1. Introducción y Contexto	2
1.1. El Problema de los Enfoques Actuales	2
1.2. Nuestra Propuesta: Morfogénesis Física	2
2. Fundamentos Matemáticos	2
2.1. Sistema de Reacción-Difusión Clásico	2
2.2. Cinética de Gierer-Meinhardt	2
2.2.1. Estado Estacionario Homogéneo	3
2.3. El Laplaciano Anisotrópico Modificado	3
2.3.1. Interpretación Física	3
2.4. Ecuación Completa del Sistema	3
3. Discretización Numérica	3
3.1. Discretización Espacial	3
3.1.1. Laplaciano Isotrópico (5-point stencil)	3
3.1.2. Laplaciano Anisotrópico	4
3.1.3. Laplaciano con Campo de Anisotropía Espacial	4
3.2. Discretización Temporal: Euler Explícito	4
3.3. Estabilidad Numérica: Condición CFL	4
4. Gradientes Radiales de Parámetros	4
4.1. Motivación Biológica	4
4.2. Campo Radial de ρ	5
4.2.1. Efecto sobre la Morfología	5
4.3. Campo Radial de Anisotropía	5
5. Inicialización Biomimética: Shark-Seed	5
5.1. Fundamento Biológico	5
5.2. Formulación Matemática	6
5.3. Comparación: Ruido Global vs Shark-Seed	6
6. Algoritmo Completo	7
6.1. Pseudocódigo Principal	7
6.2. Funciones Auxiliares	7
7. Calibración Biomédica	7
7.1. Métricas Objetivo	7
7.2. Longitud de Onda Característica	8
7.2.1. Calibración de λ a Tb.Sp	8
7.3. Grado de Anisotropía	8
7.4. Fracción de Volumen Óseo	8
7.5. Protocolo de Calibración	8
8. Implementación en Código	9
8.1. Estructura de Clases Propuesta	9
8.2. Implementación del Laplaciano Anisotrópico	9
8.3. Implementación del Shark-Seed Initializer	10
8.4. Implementación del Gradiente Radial	10

9. Validación y Tests	11
9.1. Tests de Regresión	11
9.2. Validación Biomédica	11
10. Referencias Bibliográficas	11
10.1. Fundamentos Teóricos	11
10.2. Morfología Fúngica	12
10.3. Biomecánica Ósea	12
10.4. Inicialización Biomimética	12
10.5. Ingeniería de Tejidos	12
10.6. Validación Micro-CT	12
A. Tabla de Parámetros de Referencia	13
B. Diagrama de Flujo del Algoritmo	13

1. Introducción y Contexto

1.1. El Problema de los Enfoques Actuales

La mayoría de los enfoques actuales en bioingeniería para diseño de scaffolds y metamateriales utilizan:

1. **Algoritmos estocásticos genéricos:** Ruido Perlin, Simplex, Worley
2. **Retículas matemáticas:** Giroides (TPMS), Schwarz-P, estructuras lattice
3. **Optimización topológica:** SIMP, level-set methods

Estos métodos *imitan la apariencia visual* pero no garantizan que la microestructura esté **alineada con las fuerzas mecánicas** del entorno biológico.

1.2. Nuestra Propuesta: Morfogénesis Física

Proponemos un sistema de **Reacción-Difusión Anisotrópico** basado en Gierer-Meinhardt (1972) donde:

- El **tensor de difusión** $[\alpha_x, \alpha_y]$ actúa como proxy de carga mecánica
- Los **gradientes radiales** de parámetros simulan transiciones corteza→médula
- La **inicialización tipo tiburón** garantiza continuidad topológica

2. Fundamentos Matemáticos

2.1. Sistema de Reacción-Difusión Clásico

El modelo clásico de Turing para formación de patrones se describe mediante:

$$\begin{aligned}\frac{\partial u}{\partial t} &= D_u \nabla^2 u + f(u, v) \\ \frac{\partial v}{\partial t} &= D_v \nabla^2 v + g(u, v)\end{aligned}\tag{1}$$

donde:

- $u(\mathbf{x}, t)$: Concentración del **activador**
- $v(\mathbf{x}, t)$: Concentración del **inhibidor**
- D_u, D_v : Coeficientes de difusión ($D_v \gg D_u$ para inestabilidad de Turing)
- f, g : Términos de reacción (cinética)

2.2. Cinética de Gierer-Meinhardt

Definición 2.1 (Modelo Gierer-Meinhardt). La cinética de Gierer-Meinhardt (1972) se define como:

$$f(u, v) = \rho \left(a - bu + \frac{u^2}{v + \epsilon} \right)\tag{2}$$

$$g(u, v) = \rho (u^2 - v)$$

donde $\epsilon > 0$ es un parámetro de regularización numérica.

Observación 2.1 (Ventaja sobre Schnakenberg). Gierer-Meinhardt produce “**picos**” **localizados** (columnas estructurales) en lugar de las manchas suaves de Schnakenberg. Esto es ideal para simular trabéculas óseas.

2.2.1. Estado Estacionario Homogéneo

El punto fijo del sistema se obtiene resolviendo $f(u_0, v_0) = g(u_0, v_0) = 0$:

$$\boxed{u_0 = \frac{a+1}{b}, \quad v_0 = u_0^2 = \left(\frac{a+1}{b}\right)^2} \quad (3)$$

2.3. El Laplaciano Anisotrópico Modificado

Definición 2.2 (Laplaciano Anisotrópico). Reemplazamos el operador de Laplace isotrópico estándar por un **Laplaciano Anisotrópico**:

$$\nabla_{\text{aniso}}^2 = \alpha_x \frac{\partial^2}{\partial x^2} + \alpha_y \frac{\partial^2}{\partial y^2} \quad (4)$$

donde $\alpha_x, \alpha_y \in \mathbb{R}^+$ son los **coeficientes de anisotropía direccional**.

2.3.1. Interpretación Física

El tensor de difusión $\alpha = [\alpha_x, \alpha_y]$ tiene interpretación física directa:

Condición	Efecto Físico	Morfología
$\alpha_y > \alpha_x$	Difusión preferencial vertical	Láminas verticales
$\alpha_y \approx \alpha_x$	Difusión isotrópica	Poros hexagonales
$\alpha_y < \alpha_x$	Difusión preferencial horizontal	Láminas horizontales

Tabla 1: Interpretación del tensor de anisotropía

2.4. Ecuación Completa del Sistema

Teorema 2.1 (Sistema Gierer-Meinhardt Anisotrópico). El sistema completo con anisotropía y gradientes espaciales se escribe:

$$\boxed{\begin{aligned} \frac{\partial u}{\partial t} &= D_u \left(\alpha_x(\mathbf{x}) \frac{\partial^2 u}{\partial x^2} + \alpha_y(\mathbf{x}) \frac{\partial^2 u}{\partial y^2} \right) + \rho(\mathbf{x}) \left(a - bu + \frac{u^2}{v + \epsilon} \right) \\ \frac{\partial v}{\partial t} &= D_v \left(\alpha_x(\mathbf{x}) \frac{\partial^2 v}{\partial x^2} + \alpha_y(\mathbf{x}) \frac{\partial^2 v}{\partial y^2} \right) + \rho(\mathbf{x}) (u^2 - v) \end{aligned}} \quad (5)$$

donde $\alpha_x(\mathbf{x}), \alpha_y(\mathbf{x}), \rho(\mathbf{x})$ son **campos espaciales** que permiten variación local de propiedades.

3. Discretización Numérica

3.1. Discretización Espacial

3.1.1. Laplaciano Isotrópico (5-point stencil)

Para el caso isotrópico, usamos diferencias finitas centradas:

$$\nabla^2 u|_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{(\Delta x)^2} \quad (6)$$

3.1.2. Laplaciano Anisotrópico

Para el caso anisotrópico con α_x, α_y :

$$\nabla_{\text{aniso}}^2 u|_{i,j} \approx \frac{\alpha_x(u_{i+1,j} + u_{i-1,j} - 2u_{i,j}) + \alpha_y(u_{i,j+1} + u_{i,j-1} - 2u_{i,j})}{(\Delta x)^2} \quad (7)$$

Observación 3.1 (Implementación Eficiente). Usando operaciones de roll (desplazamiento circular) de NumPy:

```
1 def laplacian_aniso(Z, alpha_y, alpha_x, dx=1.0):
2     dy = (np.roll(Z, 1, axis=0) + np.roll(Z, -1, axis=0) - 2*Z) * alpha_y
3     dx_term = (np.roll(Z, 1, axis=1) + np.roll(Z, -1, axis=1) - 2*Z) * alpha_x
4     return (dy + dx_term) / dx**2
```

3.1.3. Laplaciano con Campo de Anisotropía Espacial

Cuando $\alpha_x(\mathbf{x}), \alpha_y(\mathbf{x})$ son campos:

$$\nabla_{\text{aniso}}^2 u|_{i,j} \approx \frac{\alpha_{x,i,j}(u_{i+1,j} + u_{i-1,j} - 2u_{i,j}) + \alpha_{y,i,j}(u_{i,j+1} + u_{i,j-1} - 2u_{i,j})}{(\Delta x)^2} \quad (8)$$

3.2. Discretización Temporal: Euler Explícito

Usamos el método de Euler hacia adelante:

$$\begin{aligned} u_{i,j}^{n+1} &= u_{i,j}^n + \Delta t [D_u \nabla_{\text{aniso}}^2 u_{i,j}^n + f(u_{i,j}^n, v_{i,j}^n)] \\ v_{i,j}^{n+1} &= v_{i,j}^n + \Delta t [D_v \nabla_{\text{aniso}}^2 v_{i,j}^n + g(u_{i,j}^n, v_{i,j}^n)] \end{aligned} \quad (9)$$

3.3. Estabilidad Numérica: Condición CFL

Teorema 3.1 (Condición CFL para Difusión Anisotrópica). Para garantizar estabilidad del esquema explícito, el paso temporal debe satisfacer:

$$\Delta t \leq \frac{(\Delta x)^2}{4D_{\text{máx}} \cdot \text{máx}(\alpha_x, \alpha_y)} \cdot \eta \quad (10)$$

donde:

- $D_{\text{máx}} = \text{máx}(D_u, D_v)$
- $\eta \in (0, 1)$ es un factor de seguridad (típicamente $\eta = 0,8$)

Demostración. La condición CFL estándar para difusión 2D es $\Delta t \leq \frac{(\Delta x)^2}{4D}$. La anisotropía escala efectivamente la difusión máxima por $\text{máx}(\alpha_x, \alpha_y)$. El factor $\eta < 1$ proporciona margen para la rigidez de los términos de reacción. $\square$

4. Gradientes Radiales de Parámetros

4.1. Motivación Biológica

La estructura ósea presenta un **gradiente natural** desde la corteza (densa) hacia la médula (porosa). Para simular esto, necesitamos campos de parámetros que varíen espacialmente.

4.2. Campo Radial de ρ

Definición 4.1 (Gradiente Radial Sigmoideo). Definimos el campo de tasa de reacción como:

$$\rho(r) = \rho_{\text{centro}} + (\rho_{\text{borde}} - \rho_{\text{centro}}) \cdot \sigma\left(\frac{r - r_0}{w}\right) \quad (11)$$

donde:

- $r = \sqrt{(x - x_c)^2 + (y - y_c)^2}$ es la distancia al centro
- $\sigma(z) = \frac{1}{1+e^{-z}}$ es la función sigmoide
- r_0 es el radio de transición
- w es el ancho de la transición

4.2.1. Efecto sobre la Morfología

Región	ρ local	Densidad	Análogo Biológico
Centro ($r \rightarrow 0$)	ρ_{centro} (alto)	Alta	Corteza ósea (BV/TV ~ 90 %)
Borde ($r \rightarrow R$)	ρ_{borde} (bajo)	Baja	Hueso esponjoso (BV/TV ~ 30 %)

Tabla 2: Mapeo de $\rho(r)$ a estructura ósea

4.3. Campo Radial de Anisotropía

De manera análoga, podemos definir campos para $\alpha_y(r), \alpha_x(r)$:

$$\begin{aligned} \alpha_y(r) &= \alpha_{y,\text{centro}} + (\alpha_{y,\text{borde}} - \alpha_{y,\text{centro}}) \cdot \sigma\left(\frac{r - r_0}{w}\right) \\ \alpha_x(r) &= \alpha_{x,\text{centro}} + (\alpha_{x,\text{borde}} - \alpha_{x,\text{centro}}) \cdot \sigma\left(\frac{r - r_0}{w}\right) \end{aligned} \quad (12)$$

Esto permite simular:

- **Corteza:** Alta anisotropía ($\alpha_y/\alpha_x \approx 3$) $\rightarrow$ trabéculas alineadas
- **Esponjoso:** Baja anisotropía ($\alpha_y/\alpha_x \approx 1,2$) $\rightarrow$ poros más isotrópicos

5. Inicialización Biomimética: Shark-Seed

5.1. Fundamento Biológico

Cooper et al. (2018) demostraron que los **dentículos de tiburón** se forman mediante un mecanismo de Turing que se inicia desde puntos de nucleación y se propaga radialmente. Esto garantiza:

- **Continuidad topológica absoluta:** Sin “costuras” ni defectos de grano
- **Control de fase:** El patrón hereda la orientación de la semilla
- **Gradiente natural:** La densidad decrece desde el centro

5.2. Formulación Matemática

Definición 5.1 (Inicialización Shark-Seed). En lugar de ruido aleatorio global, inicializamos con una perturbación Gaussiana localizada:

$$u(\mathbf{x}, 0) = u_0 + A \exp \left(-\frac{|\mathbf{x} - \mathbf{x}_c|^2}{2\sigma_s^2} \right) \quad (13)$$

donde:

- $\mathbf{x}_c$ es el centro de la semilla (típicamente el centro del dominio)
- A es la amplitud de la perturbación
- σ_s es el radio de la semilla

5.3. Comparacion: Ruido Global vs Shark-Seed

Característica	Ruido Global	Shark-Seed
Nucleacion	Simultanea en todo el dominio	Desde punto central
Propagacion	No hay (estatica)	Radial (onda viajera)
Defectos de grano	Frecuentes en interfaces	Ninguno
Control de fase	Aleatorio	Determinista
Gradiente natural	No	Si (radial)
Continuidad topologica	Parcial	Absoluta

Tabla 3: Comparacion de estrategias de inicializacion

RUIDO GLOBAL		SHARK-SEED
+-----+		+-----+
.		
.		***
.		** **
.	vs	* (0) *
.		** **
.		***
+-----+		+-----+
(defectos de grano)		(propagacion radial)

6. Algoritmo Completo

6.1. Pseudocódigo Principal

Algorithm 1 Simulador Gierer-Meinhardt Anisotrópico con Gradientes

Require: N (tamaño de grilla), T (tiempo total), parámetros cinéticos y de anisotropía

Ensure: Campos u, v finales

```

1: // Fase 1: Inicialización
2:  $u_0, v_0 \leftarrow \text{CalcularEstadoEstacionario}(a, b)$  ▷ Ec. 3
3:  $u, v \leftarrow \text{SharkSeedInit}(N, u_0, v_0, \sigma_s, A)$  ▷ Ec. 13
4: // Fase 2: Campos de Gradiente
5:  $\rho_{ij} \leftarrow \text{GenerarCampoRadial}(\rho_{\text{centro}}, \rho_{\text{borde}})$  ▷ Ec. 11
6:  $\alpha_{y,ij}, \alpha_{x,ij} \leftarrow \text{GenerarCamposAnisotropia}()$  ▷ Ec. 12
7: // Fase 3: Calcular  $\Delta t$  seguro
8:  $\Delta t \leftarrow \text{CalcularCFL}(D_u, D_v, \Delta x, \text{máx}(\alpha_y), \text{máx}(\alpha_x))$  ▷ Ec. 10
9:  $n_{\text{steps}} \leftarrow \lceil T/\Delta t \rceil$ 
10: // Fase 4: Integración Temporal
11: for  $n = 1$  to  $n_{\text{steps}}$  do
12:    $\nabla_u^2 \leftarrow \text{LaplacianoAnisotropico}(u, \alpha_{y,ij}, \alpha_{x,ij})$  ▷ Ec. 8
13:    $\nabla_v^2 \leftarrow \text{LaplacianoAnisotropico}(v, \alpha_{y,ij}, \alpha_{x,ij})$ 
14:    $f_{ij} \leftarrow \rho_{ij} \cdot (a - b \cdot u_{ij} + u_{ij}^2/(v_{ij} + \epsilon))$ 
15:    $g_{ij} \leftarrow \rho_{ij} \cdot (u_{ij}^2 - v_{ij})$ 
16:    $u \leftarrow u + \Delta t \cdot (D_u \cdot \nabla_u^2 + f)$ 
17:    $v \leftarrow v + \Delta t \cdot (D_v \cdot \nabla_v^2 + g)$ 
18:    $u \leftarrow \text{máx}(u, 0)$  ▷ Clipping para estabilidad
19:    $v \leftarrow \text{máx}(v, \epsilon)$ 
20: end for
21: return  $u, v$ 

```

6.2. Funciones Auxiliares

Algorithm 2 Laplaciano Anisotrópico con Campo Espacial

```

1: function LAPLACIANOANISOTROPICO( $Z, \alpha_y, \alpha_x, \Delta x$ )
2:    $Z_{\text{up}} \leftarrow \text{roll}(Z, +1, \text{axis} = 0)$  ▷ Vecino superior
3:    $Z_{\text{down}} \leftarrow \text{roll}(Z, -1, \text{axis} = 0)$  ▷ Vecino inferior
4:    $Z_{\text{left}} \leftarrow \text{roll}(Z, +1, \text{axis} = 1)$  ▷ Vecino izquierdo
5:    $Z_{\text{right}} \leftarrow \text{roll}(Z, -1, \text{axis} = 1)$  ▷ Vecino derecho
6:    $dy \leftarrow (Z_{\text{up}} + Z_{\text{down}} - 2Z) \odot \alpha_y$  ▷  $\odot$ : producto elemento a elemento
7:    $dx \leftarrow (Z_{\text{left}} + Z_{\text{right}} - 2Z) \odot \alpha_x$ 
8:   return  $(dy + dx)/(\Delta x)^2$ 
9: end function

```

7. Calibración Biomédica

7.1. Métricas Objetivo

Para validar que la simulación reproduce estructuras óseas reales, debemos calibrar contra métricas de Micro-CT:

Métrica Ósea	Símbolo	Rango Biológico	Parámetro Algorítmico
Separación Trabecular	Tb.Sp	600–800 μm	λ (longitud de onda)
Grosor Trabecular	Tb.Th	100–200 μm	Ancho del pico de u
Grado de Anisotropía	DA	1.5–2.0	α_y/α_x
Fracción de Volumen	BV/TV	30–90 %	ρ local

Tabla 4: Mapeo de métricas óseas a parámetros del algoritmo

7.2. Longitud de Onda Característica

Proposición 7.1 (Estimación de λ). Para el sistema Gierer-Meinhardt, la longitud de onda más inestable se aproxima por:

$$\lambda \sim 2\pi \sqrt{\frac{D_v}{D_u}} \cdot \Delta x \quad (14)$$

Esta es una aproximación heurística; la relación exacta depende de los parámetros cinéticos.

7.2.1. Calibración de λ a Tb.Sp

Dado el factor de escala $s = 8,7 \mu\text{m}/\text{px}$ (calibrado con *Fomes fomentarius*):

$$\text{Tb.Sp} = \lambda_{\text{sim}} \cdot s \quad (15)$$

Para $\text{Tb.Sp} = 700 \mu\text{m}$ objetivo:

$$\lambda_{\text{target}} = \frac{700}{8,7} \approx 80 \text{ px} \quad (16)$$

7.3. Grado de Anisotropía

Definición 7.1 (Índice de Anisotropía). El grado de anisotropía (DA) se relaciona con el ratio de coeficientes:

$$\text{DA} \approx \frac{\alpha_y}{\alpha_x} \quad (17)$$

Un DA de 1.8 (típico de vértebra lumbar) requiere $\alpha_y/\alpha_x \approx 1,8$.

7.4. Fracción de Volumen Óseo

La fracción BV/TV se controla mediante $\rho(r)$:

- ρ alto $\rightarrow$ más picos de $u \rightarrow$ mayor BV/TV
- ρ bajo $\rightarrow$ menos picos $\rightarrow$ menor BV/TV

El umbral de binarización θ también afecta:

$$\text{BV/TV} = \frac{\sum_{i,j} \mathbf{1}[u_{ij} > \theta]}{N^2} \quad (18)$$

7.5. Protocolo de Calibración

1. **Paso 1:** Fijar D_v/D_u para obtener $\lambda \approx \lambda_{\text{target}}$
2. **Paso 2:** Ajustar α_y/α_x para DA objetivo
3. **Paso 3:** Calibrar $\rho_{\text{centro}}, \rho_{\text{borde}}$ para gradiente BV/TV
4. **Paso 4:** Afinar b si Tb.Th no coincide
5. **Paso 5:** Validar con datos Micro-CT reales

8. Implementación en Código

8.1. Estructura de Clases Propuesta

```
1 from abc import ABC, abstractmethod
2 import numpy as np
3 from typing import Tuple, Optional
4
5 class BaseLaplacian(ABC):
6     """Interfaz para operadores Laplacianos."""
7
8     @abstractmethod
9     def compute(self, field: np.ndarray, dx: float = 1.0) -> np.ndarray:
10         """Calcula el Laplaciano del campo."""
11         pass
12
13     @abstractmethod
14     def get_cfl_factor(self) -> float:
15         """Retorna el factor para calculo CFL."""
16         pass
17
18 class BaseInitializer(ABC):
19     """Interfaz para inicializadores."""
20
21     @abstractmethod
22     def initialize(self, grid_size: int, u_steady: float, v_steady: float,
23                   random_seed: Optional[int] = None) -> Tuple[np.ndarray, np.
24 ndarray]:
25         """Genera condiciones iniciales (U, V)."""
26         pass
27
28 class BaseGradientField(ABC):
29     """Interfaz para campos de gradiente espacial."""
30
31     @abstractmethod
32     def get_rho_field(self, grid_size: int) -> np.ndarray:
33         """Retorna campo rho(x,y)."""
34         pass
35
36     @abstractmethod
37     def get_alpha_field(self, grid_size: int) -> Tuple[np.ndarray, np.ndarray]:
38         """Retorna campos (alpha_y(x,y), alpha_x(x,y))."""
39         pass
```

Listing 1: Interfaces base para el sistema modular

8.2. Implementación del Laplaciano Anisotrópico

```
1 class AnisotropicLaplacian(BaseLaplacian):
2     """Laplaciano con anisotropia constante."""
3
4     def __init__(self, alpha_y: float = 1.0, alpha_x: float = 1.0):
5         self.alpha_y = alpha_y
6         self.alpha_x = alpha_x
7
8     def compute(self, field: np.ndarray, dx: float = 1.0) -> np.ndarray:
9         # Diferencias finitas con condiciones de borde periodicas
10         dy = (np.roll(field, 1, axis=0) + np.roll(field, -1, axis=0)
11              - 2 * field) * self.alpha_y
12         dx_term = (np.roll(field, 1, axis=1) + np.roll(field, -1, axis=1)
13                  - 2 * field) * self.alpha_x
14         return (dy + dx_term) / dx**2
```

```

15
16 def get_cfl_factor(self) -> float:
17     return max(self.alpha_y, self.alpha_x)

```

Listing 2: Clase AnisotropicLaplacian

8.3. Implementación del Shark-Seed Initializer

```

1 class SharkSeedInitializer(BaseInitializer):
2     """Inicializacion con semilla central (Cooper et al. 2018)."""
3
4     def __init__(self, seed_radius: float = 5.0, seed_amplitude: float = 0.5,
5                 n_seeds: int = 1):
6         self.seed_radius = seed_radius
7         self.seed_amplitude = seed_amplitude
8         self.n_seeds = n_seeds
9
10    def initialize(self, grid_size: int, u_steady: float, v_steady: float,
11                 random_seed: Optional[int] = None) -> Tuple[np.ndarray, np.
12                 ndarray]:
13        if random_seed is not None:
14            np.random.seed(random_seed)
15
16        U = np.full((grid_size, grid_size), u_steady)
17        V = np.full((grid_size, grid_size), v_steady)
18
19        # Semilla Gaussiana central
20        cy, cx = grid_size // 2, grid_size // 2
21        Y, X = np.ogrid[:grid_size, :grid_size]
22        r2 = (X - cx)**2 + (Y - cy)**2
23
24        seed = self.seed_amplitude * np.exp(-r2 / (2 * self.seed_radius**2))
25        U += seed
26
27        return U, V

```

Listing 3: Clase SharkSeedInitializer

8.4. Implementación del Gradiente Radial

```

1 class RadialGradientField(BaseGradientField):
2     """Campo de gradiente radial para transicion corteza-medula."""
3
4     def __init__(self,
5                 rho_center: float = 5.0,
6                 rho_edge: float = 0.2,
7                 alpha_y_center: float = 3.0,
8                 alpha_y_edge: float = 1.2,
9                 alpha_x_center: float = 1.0,
10                 alpha_x_edge: float = 1.0,
11                 transition_width: float = 0.15):
12         self.rho_center = rho_center
13         self.rho_edge = rho_edge
14         self.alpha_y_center = alpha_y_center
15         self.alpha_y_edge = alpha_y_edge
16         self.alpha_x_center = alpha_x_center
17         self.alpha_x_edge = alpha_x_edge
18         self.transition_width = transition_width
19
20     def _sigmoid_transition(self, r_norm: np.ndarray) -> np.ndarray:
21         """Funcion de transicion sigmoidal."""

```

```

22         return 1 / (1 + np.exp(-(r_norm - 0.5) / self.transition_width))
23
24     def _compute_radial_distance(self, grid_size: int) -> np.ndarray:
25         """Calcula distancia radial normalizada [0, 1]."""
26         cy, cx = grid_size // 2, grid_size // 2
27         Y, X = np.ogrid[:grid_size, :grid_size]
28         r = np.sqrt((X - cx)**2 + (Y - cy)**2)
29         return r / (grid_size // 2)
30
31     def get_rho_field(self, grid_size: int) -> np.ndarray:
32         r_norm = self._compute_radial_distance(grid_size)
33         t = self._sigmoid_transition(r_norm)
34         return self.rho_center + (self.rho_edge - self.rho_center) * t
35
36     def get_alpha_field(self, grid_size: int) -> Tuple[np.ndarray, np.ndarray]:
37         r_norm = self._compute_radial_distance(grid_size)
38         t = self._sigmoid_transition(r_norm)
39
40         alpha_y = self.alpha_y_center + (self.alpha_y_edge - self.alpha_y_center
41 ) * t
42         alpha_x = self.alpha_x_center + (self.alpha_x_edge - self.alpha_x_center
43 ) * t
44
45         return alpha_y, alpha_x

```

Listing 4: Clase RadialGradientField

9. Validación y Tests

9.1. Tests de Regresión

1. **Test de Isotropía:** Con $\alpha_y = \alpha_x = 1$, el sistema debe reproducir los resultados del simulador original (Fomes fomentarius: $\lambda \approx 46$ px).
2. **Test de Transición Topológica:** Replicar exactamente los resultados de `test_laminillas.py`:
 - Poros ($\alpha_y = 1, \alpha_x = 1$): $\chi > 0$
 - Láminas ($\alpha_y = 4, \alpha_x = 0,5$): Bandas verticales
 - Laberintos ($\alpha_y = 2, \alpha_x = 0,7$): $\chi \approx 0$
3. **Test de Shark-Seed:** Verificar que el patrón se propaga radialmente sin defectos de grano.
4. **Test de Gradiente:** Confirmar que BV/TV varía de $\sim 90\%$ (centro) a $\sim 30\%$ (borde).

9.2. Validación Biomédica

1. Comparar λ_{sim} con Tb.Sp de datos Micro-CT
2. Verificar DA simulado vs DA medido
3. Validar perfil radial de BV/TV
4. Comprobar alineación trabecular (Ley de Wolff)

10. Referencias Bibliográficas

10.1. Fundamentos Teóricos

1. Turing, A. M. (1952). The chemical basis of morphogenesis. *Phil. Trans. R. Soc. B*, 237(641), 37-72.

2. Gierer, A., & Meinhardt, H. (1972). A theory of biological pattern formation. *Kybernetik*, 12(1), 30-39.
3. Murray, J. D. (2003). *Mathematical Biology II: Spatial Models and Biomedical Applications*. Springer.
4. Cross, M. C., & Hohenberg, P. C. (1993). Pattern formation outside of equilibrium. *Rev. Mod. Phys.*, 65(3), 851-1112.

10.2. Morfología Fúngica

5. Kuhar, F., et al. (2022). The LALIP hypothesis: Pattern formation features might explain the evolution of hymenophore in Polyporales. *Theory in Biosciences*, 141, 225-238.
6. Klemm, D., et al. (2024). Hierarchical structure and mechanical properties of the tinder fungus *Fomes fomentarius*. *PLOS ONE*, 19(1), e0295432.
7. Galipot, P., et al. (2025). And growth on form? Evidence for the role of mechanical stress in hymenophore morphogenesis. *Biology*, 14(1), 45.

10.3. Biomecánica Ósea

8. Wolff, J. (1892). *Das Gesetz der Transformation der Knochen*. Hirschwald, Berlin.
9. Whitehouse, W. J. (1974). The quantitative morphology of anisotropic trabecular bone. *J. Microscopy*, 101(2), 153-168.
10. Kivell, T. L. (2016). A review of trabecular bone functional adaptation. *J. Anatomy*, 228(4), 569-594.
11. Gibson, L. J., & Ashby, M. F. (1997). *Cellular Solids: Structure and Properties*. Cambridge University Press.

10.4. Inicialización Biomimética

12. Cooper, R. L., et al. (2018). An ancient Turing-like patterning mechanism regulates skin denticle development in sharks. *Science Advances*, 4(11), eaau5484.
13. Sick, S., et al. (2006). WNT and DKK determine hair follicle spacing through a reaction-diffusion mechanism. *Science*, 314(5804), 1447-1450.

10.5. Ingeniería de Tejidos

14. Hollister, S. J. (2005). Porous scaffold design for tissue engineering. *Nature Materials*, 4(7), 518-524.
15. Hutmacher, D. W. (2000). Scaffolds in tissue engineering bone and cartilage. *Biomaterials*, 21(24), 2529-2543.
16. Melchels, F. P. W., et al. (2010). Mathematically defined tissue engineering scaffold architectures. *Biomaterials*, 31(27), 6909-6916.

10.6. Validación Micro-CT

17. Buxsein, M. L., et al. (2010). Guidelines for assessment of bone microstructure in rodents using micro-computed tomography. *J. Bone Miner. Res.*, 25(7), 1468-1486.
18. Harrigan, T. P., & Mann, R. W. (1984). Characterization of microstructural anisotropy in orthotropic materials. *J. Materials Science*, 19(3), 761-767.

A. Tabla de Parámetros de Referencia

Parámetro	Símbolo	Rango	Default
<i>Difusión</i>			
Coef. difusión activador	D_u	0.1–10	1.0
Ratio difusión	D_v/D_u	50–5000	150
<i>Cinética Gierer-Meinhardt</i>			
Término fuente	a	0.01–1.0	0.1
Saturación	b	0.5–5.0	1.0
Tasa de reacción	ρ	0.1–10.0	0.2
Regularización	ϵ	10^{-8} – 10^{-4}	10^{-6}
<i>Anisotropía</i>			
Coef. anisotropía Y	α_y	0.5–5.0	1.0
Coef. anisotropía X	α_x	0.5–5.0	1.0
<i>Gradiente Radial</i>			
ρ centro (corteza)	ρ_c	2.0–10.0	5.0
ρ borde (esponjoso)	ρ_e	0.1–1.0	0.2
Ancho transición	w	0.05–0.3	0.15
<i>Inicialización Shark-Seed</i>			
Radio semilla	σ_s	3–20 px	10
Amplitud semilla	A	0.1–1.0	0.5
<i>Numéricos</i>			
Tamaño de grilla	N	256–2048	512
Resolución espacial	Δx	0.5–2.0	1.0
Factor seguridad CFL	η	0.5–0.9	0.8

Tabla 5: Tabla completa de parámetros del sistema

B. Diagrama de Flujo del Algoritmo

El algoritmo principal sigue el siguiente flujo:

1. **Inicio:** Cargar configuración y parametros
2. **Inicializacion:** Calcular estado estacionario (u_0, v_0) y aplicar Shark-Seed
3. **Campos de Gradiente:** Generar $\rho(r)$, $\alpha_y(r)$, $\alpha_x(r)$
4. **CFL:** Calcular Δt seguro segun Ec. 10
5. **Bucle temporal** (mientras $t < T$):
 - Calcular Laplaciano anisotropico de u y v
 - Evaluar reacciones $f(u, v)$, $g(u, v)$ con ρ local
 - Actualizar campos via Euler explicito
 - Aplicar clipping: $u \geq 0$, $v \geq \epsilon$
 - Incrementar $t \leftarrow t + \Delta t$
6. **Fin:** Retornar campos u , v finales

```

[Inicio] --> [Inicializar u0,v0] --> [Generar campos]
                                     |
                                     v
[Fin] <-- [t >= T?] <-- [Clipping] <-- [Euler] <-- [Reacciones]
      |                                     ^
      | (No)                             |
      +-----> [Laplaciano aniso] -----+

```